A Conceptual Primer on MeTTa-Pure, MeTTa-Core, Verified Profiles, and Lean-Based Runtime Architecture

Dated design record for future implementation and research work

Prepared as an archival technical primer

2026-03-06

Abstract

This note consolidates a coherent design picture for a clean dependent-type-theoretic kernel for MeTTa together with its relation to the broader MeTTa family of operational profiles, modal semantics, verified runtimes, and theorem-backed domain-specific profiles. The central claim is that a satisfactory architecture requires four distinct but connected layers: a small trusted kernel, a MeTTa-IL presentation of that kernel, a conservative bridge between them, and a profile-oriented runtime family in which HE, PeTTa, and future profiles are comparable without being collapsed into one semantics. The note also records a second major theme: verified implementation should proceed by declarative specification, explicit reference backends, and refinement to optimized runtimes rather than by ad hoc operational growth. The article is intentionally conceptual and is designed to function both as an archival design record and as a primer for future discussions and new implementation sessions.

Contents

1 Purpose **1**

2 The core diagnosis **1**
- 2.1 Why the ambient host representation is not the kernel theorem domain . . . 2
- 2.2 Why runtime typing and kernel typing are different things . . . 2

3 Four distinct objects **2**
- 3.1 1. The trusted kernel . . . 2
- 3.2 2. The MeTTa-IL presentation of the kernel . . . 3
- 3.3 3. The bridge . . . 3
- 3.4 4. The runtime family . . . 3

4 MeTTa-Pure is not just a DTT kernel **3**

5 The A/B/C semantic stratification **3**
- 5.1 A: executable operational fragment . . . 4
- 5.2 B: full kernel theory . . . 4
- 5.3 C: profile-side theory closure . . . 4

6 Why the old bridge failed **4**

7 The correct quotation: contextual locally nameless quotation **4**
7.1 A useful policy abstraction . . . 5

8 The bridge theorem should be about substitution, not just `inst0` **5**

9 A separate kernel syntax is the right long-term route **5**

10 MeTTa-IL as the operational and modal base **5**

11 Profiles are the right comparison unit **6**

12 PeTTa should be specified declaratively first **6**
12.1 Why this helps implementation . . . 6

13 Verified runtime architecture: reference first, optimization second **7**

14 What a verified SimplePeTTa should mean **7**

15 The right package structure **7**

16 Simple runtime, staged runtime, and external kernels **8**
16.1 Why a staged runtime is still compatible with the common core . . . 8

17 Optional DSL profiles should be theorem-backed and lowered **9**

18 What should be considered locked in **9**

19 Open problems worth keeping in view **9**
19.1 Algorithmic type checking for the kernel . . . 9
19.2 Final bridge packaging . . . 10
19.3 Intrinsic profile-side theory closure vs transported closure . . . 10
19.4 How much of broader MeTTa should be reflected into pure theory . . . 10

20 Least-regret roadmap **10**

21 Conclusion **11**

1 Purpose

The purpose of this document is to preserve the main conceptual results of an extended design investigation into MeTTa-Pure, MeTTa-Core, PeTTa, and verified runtime architecture. The goal is not to present finished metatheory or finished software, but to state clearly what has been learned, which distinctions should be considered stable, and which proof and implementation directions appear least likely to require later reversal.

The note is written as a primer. It is meant to answer the following recurring questions:

- What exactly is MeTTa-Pure?
- How is it related to MeTTa-IL, HE, PeTTa, and broader MeTTa runtimes?
- Why is a separate kernel needed at all?

- How should verified runtimes and staged compilers be organized?
- How should profile-specific notations and DSLs fit into the system without contaminating the shared core?

2 The core diagnosis

The original temptation is to identify four things that should not be identified:

1. the ambient host representation used by MeTTa-IL,
2. the trusted kernel theorem domain,
3. the executable runtime relation, and
4. the broader family identity of "being a form of MeTTa."

The design work shows that these have to be separated.

2.1 Why the ambient host representation is not the kernel theorem domain

A host datatype can be excellent for runtime lowering, interoperation, profile compilation, quotation, and modal analysis while still being the wrong place to prove kernel metatheory. In the MeTTa setting, the ambient `Pattern` syntax supports more constructors and more operational structure than the tiny DTT kernel actually intends to use. Once confluence, injectivity, or subject reduction are stated over the whole ambient host language, proofs become polluted by constructors that are not part of the kernel and were never intended to be handled by the kernel congruence rules.

This yields the first stable principle:

> **Kernel metatheory should be proved on the least syntax closed under kernel operations, not on the ambient host representation.**

2.2 Why runtime typing and kernel typing are different things

Operational MeTTa profiles may use type annotations, runtime checks, or profile-specific reasoning aids. Those are useful, but they are not automatically proof-assistant kernels. A DTT kernel needs conversion-sensitive typing and subject reduction in the kernel sense. A runtime annotation discipline may be fast and practical while still not being the right object for trusted internal reasoning.

This yields the second stable principle:

> **Operational typing and kernel typing should be related by bridge theorems, not identified by default.**

3 Four distinct objects

The clean architecture has four primary objects.

3.1 1. The trusted kernel

The trusted kernel is a scoped dependent type theory, here called *PureKernel*. It should provide:

- its own syntax, ideally indexed by scope depth,

- renaming and substitution,
- one-step reduction and reflexive-transitive closure,
- confluence or Church–Rosser style results,
- typing with conversion, and
- subject reduction.

This is the object on which proof-theoretic theorems are established.

3.2 2. The MeTTa-IL presentation of the kernel

The kernel should also have a *Pattern-side presentation* as a `LanguageDef`. This is the way the pure kernel appears inside the broader MeTTa world. It is not merely an encoding trick. It is the profile-level presentation of the kernel in the shared substrate.

3.3 3. The bridge

A kernel becomes a bona fide form of MeTTa only if there is a conservative bridge theorem from the trusted kernel into the MeTTa-IL presentation. The bridge should preserve at least:

- constructor shape,
- computational steps,
- and, eventually, typing or at least typing-relevant structure.

3.4 4. The runtime family

The broader MeTTa world should be organized as a family of *profiles*, not as one giant undifferentiated semantics. A useful conceptual interface is:

```
MeTTaCoreProfile = LanguageDef + premise program + runtime policy + optional
state constructor + profile metadata.
```

This lets one talk coherently about Pure, HE, PeTTa, and fuller runtime profiles without forcing them to be literally the same language.

4 MeTTa-Pure is not just a DTT kernel

A bare DTT kernel is not yet a form of MeTTa. It becomes one only when all of the following hold:

1. the kernel has a MeTTa-IL presentation,
2. the presentation is a valid profile in the shared family, and
3. there is a conservative bridge theorem from the kernel into that profile.

The right slogan is:

> **MeTTa-Pure is a bona fide MeTTa form exactly when it is a conservative core profile equipped with a proved embedding from the trusted kernel into the shared profile substrate.**

This is stronger than saying "the syntax looks similar" and more informative than saying "it can be encoded somehow."

5 The A/B/C semantic stratification

A major design breakthrough was recognizing that there are *three* semantic layers, not two.

5.1 A: executable operational fragment

Layer A is the currently executable fragment that is meant to run directly against the pure profile. It should be small, explicit, and directly sound into the profile's reduction machinery. It is the layer used for concrete execution and runtime tests.

5.2 B: full kernel theory

Layer B is the full kernel reduction relation: the true DTT proof-theoretic reduction used for confluence, injectivity, and subject reduction. B contains congruence structure that A may not expose directly as executable one-step behavior.

5.3 C: profile-side theory closure

Layer C is the profile-side analogue of B. It lives on quoted profile terms and closes the profile's computational base steps under the same pure contexts that B uses internally. C exists because B generally does not map directly to raw profile one-step reduction.

The stable lesson is:

> **Do not choose between A and B. Keep A for execution, keep B for proof theory, and define C so that B has a mathematically honest home on the profile side.**

6 Why the old bridge failed

The original bridge attempts failed around beta reduction and substitution under binders. The key insight is that the failure was caused by the *wrong quotation*, not by any impossibility of a faithful bridge.

A naive map from scoped kernel variables straight to host-side bound variables makes the host opening operation appear to be the same thing as kernel substitution. It is not. The host and kernel are using different binding interfaces.

This yields another stable principle:

> **The quotation must be contextual and locally nameless. Raw variable-index quotation is only a debugging view, not the semantic bridge.**

7 The correct quotation: contextual locally nameless quotation

The bridge should be parameterized by a naming environment:

$$\texttt{quoteTmWith} : (\texttt{Fin n} \to \texttt{String}) \to \texttt{PureTm n} \to \texttt{Pattern}.$$

Its intended behavior is:

- kernel variables become `fvar`s selected from the environment,

- under a binder one chooses a fresh binder name,
- the body is quoted with the extended environment,
- the chosen free variable is then closed using `closeFVar`.

This bridge is the correct way to connect scoped de Bruijn syntax with a locally nameless host representation.

7.1 A useful policy abstraction

A minimal naming policy should provide:

- a binder-name function $\nu : \mathbb{N} \to \texttt{String}$,
- injectivity of ν,
- and a compatibility predicate ensuring that future binder names are fresh relative to the current environment.

This turns ad hoc freshness side conditions into a stable interface.

8 The bridge theorem should be about substitution, not just `inst0`

A recurring proof trap is trying to prove a special-case bridge for one-substitution `inst0` before proving the generic substitution theorem. Under binders, the kernel naturally recurses using lifted substitutions, not by repeatedly invoking the special case.

The correct theorem shape is:

> quotation commutes with *general kernel substitution* after translating the substitution into a host-side substitution environment.

Once that theorem exists, the `inst0` bridge becomes a corollary rather than a primitive battle.

This is not just a proof-engineering convenience. It is the mathematically correct level of abstraction.

9 A separate kernel syntax is the right long-term route

A stable kernel should use its own scoped syntax rather than a purity predicate over a larger host datatype. A well-scoped kernel datatype has decisive advantages:

- fewer meaningless cases in proofs,
- cleaner substitution and renaming theorems,
- no need to thread ambient purity predicates through every theorem,
- and a clear boundary between trusted theory and runtime host representation.

This does *not* mean the host representation is discarded. It means the host representation becomes a target of quotation, export, and runtime interoperation rather than the theorem domain itself.

10 MeTTa-IL as the operational and modal base

MeTTa-IL should be treated as the shared operational substrate and modal/indexed base. Its main roles are:

- provide the common `Pattern`/`LanguageDef` substrate,
- support generic rewrite engines and premise handling,
- host shared runtime profile data,
- and supply derived modal/behavioral semantics such as OSLF and related categorical constructions.

This means that MeTTa-IL is *not* "the DTT itself," but it is exactly the right place for:

- runtime profiles,
- profile comparison,
- lowering targets,
- and profile-side theory closures.

11 Profiles are the right comparison unit

One should compare MeTTa-family systems as profiles over a shared substrate, not as disconnected implementations. This viewpoint resolves several confusions at once:

- Pure can be small and trusted without ceasing to be MeTTa,
- HE and PeTTa can differ operationally without ceasing to belong to the same family,
- and future profiles can be added without re-architecting the kernel.

This leads to a stable conceptual division:

- **Pure**: kernel-oriented profile,
- **HE**: runtime-oriented profile,
- **PeTTa**: logic- and Prolog-oriented profile,
- **Full/Core**: richer stateful runtime profiles.

12 PeTTa should be specified declaratively first

A second major theme is the design of a verified PeTTa runtime. The strongest conclusion is:

> **PeTTa should be specified first as a declarative goal/command calculus, and only then implemented by compilation into an executable backend.**

The right conceptual spec is three-layered:

1. a pure declarative judgment for expression evaluation,
2. a stateful declarative judgment for commands and sequencing,
3. and an operational instruction layer or goal calculus for the actual execution kernel.

This makes PeTTa intelligible both to logicians and to runtime engineers.

12.1 Why this helps implementation

Once PeTTa is viewed as:

$$\text{surface language} \to \text{declarative core} \to \text{executable backend},$$

then implementation can proceed by:

1. freezing the spec bundle,
2. building a reference backend,
3. proving soundness and completeness against the declarative spec,
4. then refining to optimized and indexed implementations.

This is exactly the verified implementation posture that avoids semantic drift.

13 Verified runtime architecture: reference first, optimization second

The clean runtime pipeline should have these layers:

1. **Declarative specification layer** (in the proof package)
2. **Reference backend** with an explicit core evaluator
3. **Session facade** that orchestrates user-facing behavior
4. **Optimized backend** that refines the reference backend
5. **Conformance layer** proving runtime-refines-spec and optimized-refines-reference

The main lesson from the runtime proof work is that the *Session* module should be only a facade/orchestrator. Search logic, deterministic strategy, nested-effect evaluation, and reference semantics belong in backend modules, not in the user-facing session wrapper.

The stable theorem target is:

```
Session.evalWithState = referenceEvalWithStateCore
```

but only after the reference backend itself has been made explicit and its preservation theorems are composed in the backend layer.

14 What a verified SimplePeTTa should mean

For SimplePeTTa, the intended notion of verification is:

- the runtime executes a frozen profile bundle,
- the reference backend is proved sound with respect to the declarative core,
- completeness is established at least for a designated terminating or benchmark fragment,
- and the optimized runtime is proved to refine the reference backend.

This provides a realistic notion of verified runtime while leaving room for bounded execution, indexing, and practical search strategies.

15 The right package structure

A stable package structure should separate:

- a lightweight shared executable core package,
- an algorithms/runtime package,
- and a proof-heavy metatheory package.

The shared core package should contain only the computational substrate:

- `Pattern`,
- `LanguageDef`,
- bindings and matching,
- premise representations,
- and the basic engine interfaces.

The runtime package should depend on that lightweight core and implement:

- simple reference runtimes,
- staged runtimes,
- and optional indexing/search backends.

The proof package should depend on the same core and prove:

- kernel metatheory,
- profile correctness,
- runtime conformance,
- and optimization refinement.

This avoids duplicating types and avoids forcing runtime builds to inherit heavyweight theorem dependencies.

16 Simple runtime, staged runtime, and external kernels

The right sequence is:

1. build a simple, directly verified runtime first,
2. then build a staged or specialized runtime as a refinement,
3. only then consider external hot kernels or low-level verified code paths.

This keeps the semantic source of truth in one place and makes performance work conservative rather than semantic.

16.1 Why a staged runtime is still compatible with the common core

A staged runtime should still consume the same frozen profile/spec bundle as the simple runtime. The difference is not semantic but operational: staging specializes dispatch, matching, indexing, and premise handling. This means the staged runtime is a refinement of the same semantics, not a second semantics.

17 Optional DSL profiles should be theorem-backed and lowered

Optional DSLs, such as a tensor/index profile, should not mutate the core runtime pipeline. They should instead follow the pattern:

1. typed surface/profile syntax,
2. semantic invariants and proof obligations,
3. lowering into the shared MeTTa-IL `Pattern` substrate,
4. and a portable runtime corpus if useful.

This yields a healthy extension discipline:

> **new notation and new profile logic should enter the system through theorem-backed lowering, not by modifying the shared runtime core.**

18 What should be considered locked in

The following decisions appear stable enough to be treated as part of the architecture rather than provisional experiments.

1. **Separate kernel syntax**: the trusted kernel should not be proved directly on ambient host syntax.
2. **Contextual quotation**: raw quotation is debugging-only; the semantic bridge must be environment-sensitive and locally nameless.
3. **A/B/C split**: executable profile steps, kernel theory steps, and profile-side theory closure are all necessary.
4. **Profiles over substrate**: Pure, HE, PeTTa, and fuller runtimes should be compared as profiles over a shared substrate.
5. **Declarative-first PeTTa**: the logic and command kernel should be specified declaratively before optimization.
6. **Reference-before-optimization**: optimized runtimes should refine a reference backend, not directly refine a textual spec.
7. **Lightweight shared core package**: shared executable substrate should be factored below the heavy proof package.
8. **Theorem-backed DSL lowering**: profile extensions should lower into the shared substrate without polluting the core runtime.

19 Open problems worth keeping in view

Several important questions remain open, but they are now clearly framed.

19.1 Algorithmic type checking for the kernel

The kernel already has the right theorem-theoretic shape, but a full algorithmic checker with soundness and completeness remains to be stabilized.

19.2 Final bridge packaging

The kernel-to-profile bridge still needs its final public theorem surface and packaging. The structural ideas are now clear: contextual quotation, substitution bridge, B-to-C theorems, and profile embedding. The remaining task is proof completion and API cleanup.

19.3 Intrinsic profile-side theory closure vs transported closure

The intrinsic contextual closure on the profile side is the mathematically correct object. Transported closures may still be useful as proof devices, but they should not replace the intrinsic theorem-level object.

19.4 How much of broader MeTTa should be reflected into pure theory

One important long-term question is how far the pure kernel should be reflected upward into richer runtime profiles, and which profile-level structures deserve direct theorem-level analogues.

20 Least-regret roadmap

A practical roadmap consistent with all the foregoing principles is:

For MeTTa-Pure

1. keep the kernel syntax and metatheory on the scoped kernel,
2. finish the generic renaming and substitution bridge,
3. derive the single-substitution bridge as a corollary,
4. complete the B-to-C bridge theorems,
5. and package the pure profile embedding cleanly.

For LeanPeTTa

1. keep the declarative spec as the source of truth,
2. expose the reference backend explicitly,
3. prove backend preservation and session-wrapper agreement,
4. prove optimized refinement only after the reference path is stable,
5. and guard performance with corpus-based threshold checks rather than by semantic shortcuts.

For the runtime family

1. keep profiles comparable via the shared substrate,
2. use frozen profile artifacts for cross-language/runtime alignment,
3. and treat optional DSLs as theorem-backed profile layers rather than runtime mutations.

21 Conclusion

The main achievement of the design work is not merely a collection of partial proofs or a collection of implementations. It is a clarified architecture.

The architecture says:

- a DTT kernel must be small and trusted,
- a bona fide form of MeTTa must exist as a profile over the shared substrate,
- the kernel and profile must be connected by an explicit conservative bridge,
- runtime verification should proceed through declarative spec, reference backend, and refinement,
- and profile-specific expressivity should be added through theorem-backed lowering rather than by eroding the shared core.

With these distinctions in place, the project no longer looks like an unresolved tangle between theorem proving, language design, and runtime engineering. It looks like a coherent layered system in which each layer has a clear job and clear interfaces to the others.

Appendix: concise glossary

Term	Meaning
PureKernel	The trusted scoped dependent type theory used for kernel metatheory.
MeTTa-IL	The shared operational substrate based on `Pattern`, `LanguageDef`, premise machinery, and derived modal/profile constructions.
Profile	A language/runtime/theory configuration over the shared substrate.
Pure profile	The MeTTa-IL presentation of the trusted kernel.
Bridge	The conservative embedding/quotation theorem connecting kernel terms to profile terms.
A layer	Executable operational/profile-facing reduction.
B layer	Full kernel reduction used for confluence, injectivity, and subject reduction.
C layer	Profile-side theory closure corresponding to the kernel theory.
Reference backend	The explicit backend whose semantics are proved against the declarative specification.
Optimized backend	A runtime meant to refine the reference backend rather than redefine semantics.
Theorem-backed lowering	The discipline of introducing DSLs or profiles by giving a typed syntax plus a proved lowering into the shared substrate.